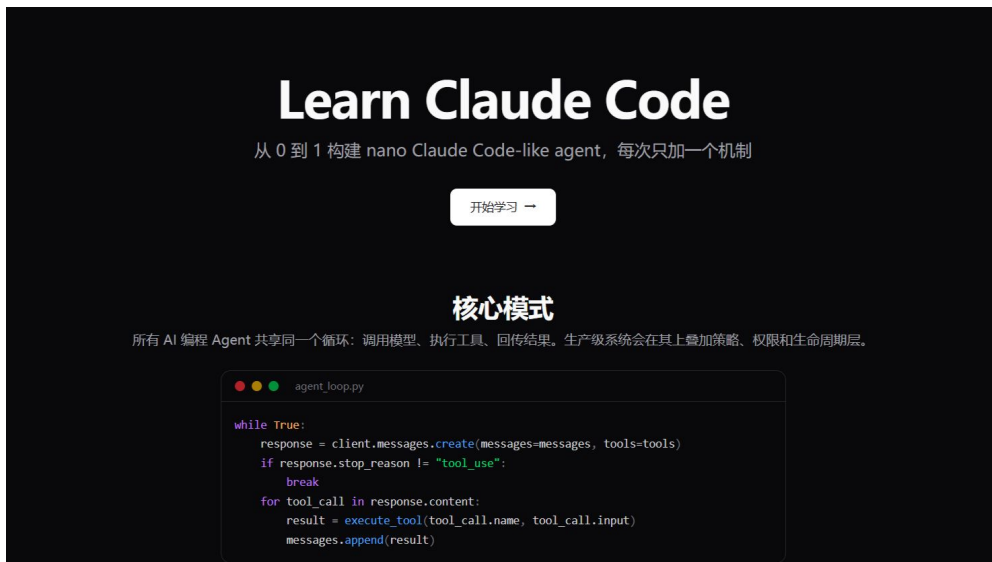


课程笔记

【Learn Claude Code 源码详解】—本地启动 Agent

开发实战分享屋

2026 年 4 月 13 日



视频作者/频道：开发实战分享屋

发布日期：2026-03-03

视频时长：6:14

视频链接：<https://www.bilibili.com/video/BV1mcAfzqEnE/>

目录

1 引言—从手动 coding 到 Agent 开发	2
1.1 当今开发模式的转变	2
1.2 现场演示：让 Claude Code 写一个并运行 Hello World	2
1.3 Agent 与聊天机器人的本质区别	3
1.4 本章小结	3
2 为什么要阅读 Agent 源码	4
2.1 抛开流行名词，直击 Agent 的运作本质	4
2.2 learn-claude-code 项目介绍与学习目标	4
2.3 精通源码意味着更深理解，意味着更好的应用	5
2.4 本章小结	5
3 环境配置与 API 密钥准备	5
3.1 跟随 README Quick Start	5
3.2 配置密钥与模型	6
3.3 从阿里云百炼获取凭证	8
3.4 本章小结	8
4 首次运行 NANO Agent 实战演示	9
4.1 运行第一个脚本	9
4.2 输入相同 prompt 进行对比	9
4.3 验证生成结果并在外部运行	11
4.4 本章小结	12
5 总结与延伸	12
5.1 本章小结	12
5.2 后续学习路径	13

1 引言—从手动 coding 到 Agent 开发

1.1 当今开发模式的转变

如果你经常关注技术圈动态，一定会频繁听到 Agent、AI coding 这样的词汇。它们并非空洞的营销名词，而是正在真切地重塑软件开发的生產方式。过去，我们习惯于自己一行一行地写代码、调试、编译、运行；而今天，越来越多的开发者开始习惯另一种工作流——直接告诉 AI 自己想做什么，由 AI 生成代码甚至自动执行。

这种转变的核心在于：开发重心正从“手写实现”迁移到“意图表达与结果验证”。对于简单的需求，AI 的响应速度往往不逊色于人类；而对于复杂的系统级任务，Agent 的优势则更为显著——它能够跨文件推理、调用工具、持续迭代，直到任务完成。

1.2 现场演示：让 Claude Code 写一个并运行 Hello World

为了让大家直观感受 Agent 的工作方式，我们不妨从一个最经典的例子开始：实现并运行一段 Hello World 的 Python 程序。

在 VS Code 的 Roo Code 插件中，只需要输入这样一句自然语言提示：

1 实现一个 `hello world` 的 Python 程序并跑起来

Roo Code 接收到指令后，并没有仅仅返回一段代码供用户手动复制粘贴，而是直接生成了 `hello_world.py` 文件，并在终端中执行了 `python hello_world.py`，最终输出了预期的结果。

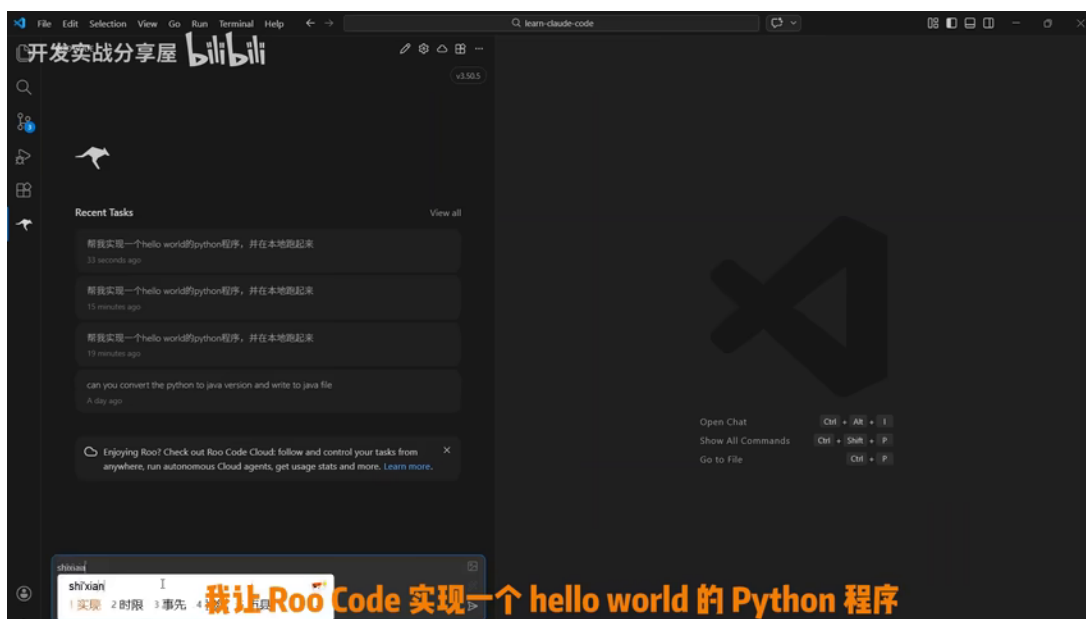


图 1: 在 VS Code 的 Roo Code 插件中输入自然语言提示，Agent 开始响应并进入工作模式¹

更令人印象深刻的是，Agent 会自动切换到 (code creation) 模式，主动完成文件的创建与运行。这在传统的聊天机器人中是无法想象的。

A

gent 不只是聊天，它会主动生成文件并执行程序。

¹视频画面时间区间：00:00:17–00:00:28

如 Figure 2 所示，整个过程中用户无需手动新建文件、粘贴代码或敲击运行命令——Agent 自己完成了从”理解意图”到”生成产物”再到”验证结果”的完整闭环。

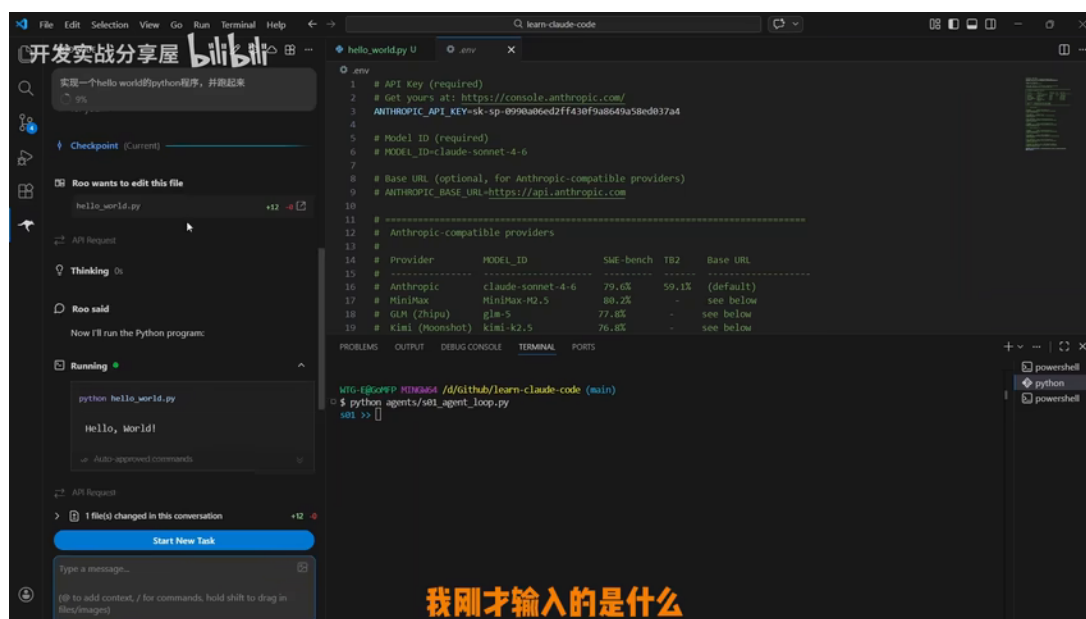


图 2: Agent 自动生成 `hello_world.py` 文件并执行，终端输出 `Hello, World!`²

对于这个最简单的例子，人类开发者当然也能很快完成；但正如视频中所强调的——”。。。但是我相信你也快不过 AI”。当任务复杂度上升时，Agent 能够同时处理多文件编辑、工具调用和错误修复，这种效率优势将呈指数级放大。

1.3 Agent 与聊天机器人的本质区别

看到这里，你可能会问：这普通的 ChatGPT、Claude 网页版聊天有什么区别？

聊

天机器人（chatbot）的核心能力是”对话”：你提问，它回答，交互止于文本。而 **Agent** 的核心能力是”行动”：它能够感知环境状态、做出决策，并通过 **Function Calling** 等机制主动调用外部工具（如文件读写、命令执行、网络请求），从而将用户的意图转化为真实世界中的变更。

换句话说，聊天机器人是”你说我听”，而 Agent 是”你说我做”。当我们说”实现一个程序并跑起来”时，Agent 不仅会写出代码，还会真正去创建文件、运行程序、检查输出——这使它成为开发者的协作伙伴，而不仅仅是问答助手。

1.4 本章小结

本章从开发范式的宏观转变切入，通过一个 Hello World 的实时演示，让大家亲身体验了 Agent 的工作方式。我们重点强调了以下几点：

- 当今开发模式正从”手写每一行代码”转向”与 AI Agent 协作”；

²视频画面时间区间：00:00:39-00:00:55

- Agent 区别于聊天机器人的关键特征在于：它会主动生成文件、执行命令并完成验证闭环；
- 简单任务中 AI 已经很快，复杂任务中 Agent 的优势将更加显著。

在下一章，我们将进一步探讨：为什么”会用工具”还不够，深入理解 Agent 的源码才能真正掌握这门新技术。

2 为什么要阅读 Agent 源码

2.1 抛开流行名词，直击 Agent 的运作本质

在第一章的演示中，我们看到 Agent 能够自动生成文件并执行代码。这背后涉及一系列令人眼花缭乱的业内术语：Skill、MCP Server、Function Calling 等等。然而，这些名词固然重要，却只是实现层面的标签。真正值得我们追问的是——Agent 究竟是如何运作的？

正如视频中讲师所强调的（00:01:13--00:01:42），抛开这些流行名词不谈，我们需要了解的是 Agent 的核心机制：它如何接收指令、如何决定是否调用工具、如何组织输入输出的循环、以及如何在无人干预的情况下完成任务。Function Calling 是大模型通过结构化输出来触发外部工具的技术；MCP Server 是为 Agent 提供标准化外部能力接口的协议服务器；Skill 则是封装好的专项能力模块。它们共同服务于一个目标——让 Agent 具备自主行动的能力。但如果不理解这些零部件如何协同工作，就只能停留在”会调接口”的层面。

2.2 learn-claude-code 项目介绍与学习目标

为了直观地回答”Agent 如何运作”这个问题，视频中推荐了一个专门用于学习 Agent 基本原理的 GitHub 仓库：

<https://github.com/shareAI-lab/learn-claude-code>

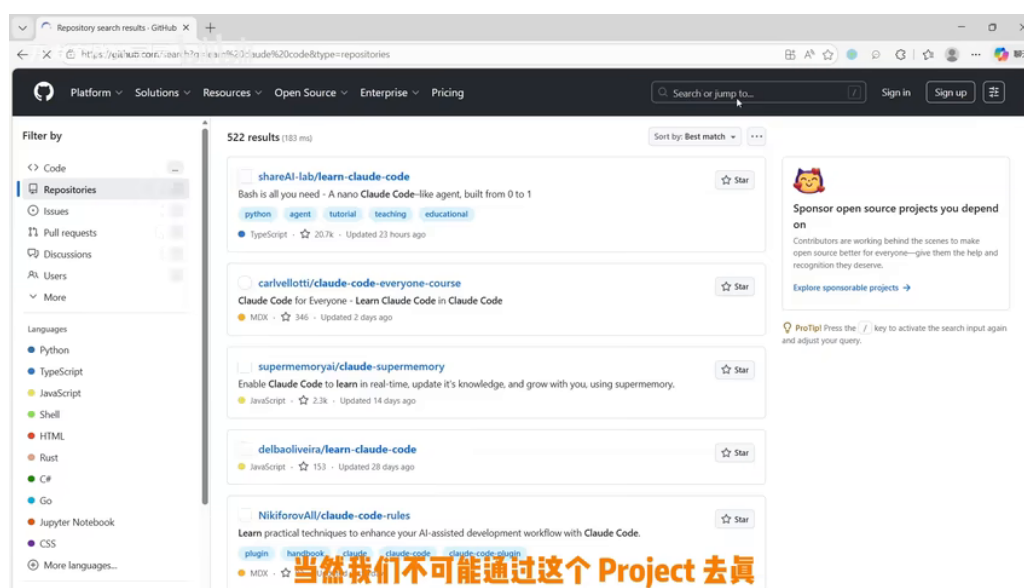


图 3: GitHub 搜索结果页面，第一个仓库为 shareAI-lab/learn-claude-code，用于学习 Agent 的基本原理³

learn-claude-code 是一个学习项目，其目的在于用最简洁的代码展示 Agent 的最基本结构。它不是生产级别的框架，也不具备企业级部署所需的高可用、安全隔离和复杂 orchestration 能力。但对于初学者而言，它是一把打开 Agent 源码大门的钥匙。

仓库的 README 中包含 Quick Start 指南，列出了克隆代码、安装依赖、配置环境变量和运行示例的命令（00:02:45--00:02:55）。如仓库 README 中的 Quick Start 所示，我们将在下一章详细展开。具体的环境配置步骤将在第三章详细讲解，第四章则带着读者运行第一个极简 Agent 示例。

2.3 精通源码意味着更深理解，意味着更好的应用

为什么要阅读源码？因为“会用”和“精通”之间，隔着的正是对原理的深刻理解。以前当我们说自己熟悉某个框架时，往往意味着我们读过它的源代码、理解过它的设计思想；如今面对 Agent 这一新兴范式，同样需要如此（00:02:15--00:02:44）。

以

前精通框架靠读源码，现在精通 Agent 也需要理解其工作原理。

一旦你了解了 Agent 最简单的结构——一个循环、若干工具定义、一次模型调用、一轮结果反馈——你就可以举一反三，去阅读更复杂、更成熟的 Agent 框架源码。这种由浅入深的路径，是掌握任何技术的通用法则。下一章，我们将从环境配置开始，迈出亲手运行这个最小 Agent 的第一步。

2.4 本章小结

本章阐明了“会用工具”与“懂原理”之间的差距。我们回顾了 Skill、MCP Server、Function Calling 等术语的含义，但始终强调：理解 Agent 的运作机制才是核心。我们介绍了用于学习的 learn-claude-code 仓库，并指出深入源码能够帮助我们在应用 Agent 时更加得心应手。接下来，我们将进入实操环节，先配置环境，再运行第一个 NANO Agent。

3 环境配置与 API 密钥准备

在正式进入源码分析之前，我们需要先把项目跑起来。本节将参考 learn-claude-code 仓库 README 中的 Quick Start，逐步完成本地环境配置与模型 API 密钥的准备工作。

3.1 跟随 README Quick Start

打开项目仓库后，第一眼就能看到 README 里提供了清晰的 Quick Start 指引。按照指引，克隆仓库、安装依赖之后，最关键的一步就是配置环境变量。

³视频画面时间区间：00:01:43-00:01:56

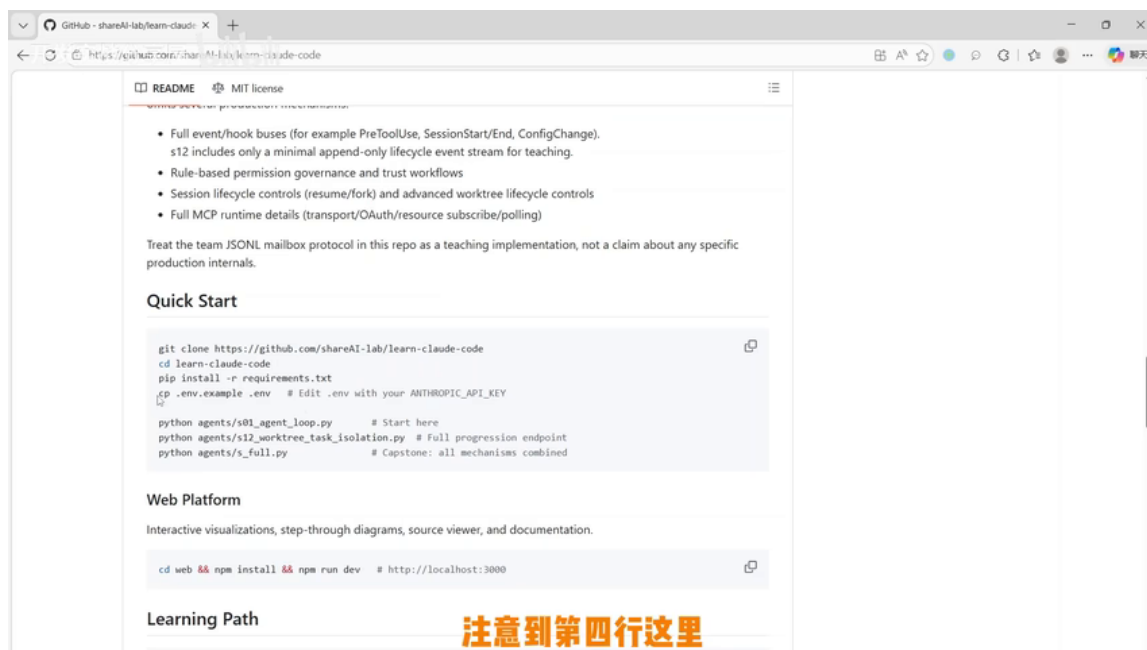


图 4: 仓库 README 中的 Quick Start 部分，列出了克隆、安装依赖、配置环境变量和运行示例的命令⁴

如Figure 4 所示，README 中的 Quick Start 条目非常简洁，只需要逐条执行即可。特别需要注意的是第四行——这里要求我们配置 Anthropic 的 API 密钥以及相关环境变量。在下一步中，我们将具体说明这三项配置的含义与写法。

3.2 配置密钥与模型

为了让 NANO Agent 能够调用大模型进行推理，我们需要提供三项基本信息：API 密钥、Base URL 和模型名称。如视频中所示，这三项通常以环境变量的形式配置在 `.env` 文件中。

⁴视频画面时间区间：00:02:45–00:02:56。

```
Get yours at: https://console.anthropic.com/
ANTHROPIC_API_KEY=sk-sp-0990a06ed2ff430f9a8649a58ed037a4

Model ID (required)
MODEL_ID=claude-sonnet-4-6

Base URL (optional, for Anthropic-compatible providers)
ANTHROPIC_BASE_URL=https://api.anthropic.com

=====
Anthropic-compatible providers
=====

Provider      MODEL_ID      SWE-bench  TB2      Base URL
-----
Anthropic      claude-sonnet-4-6  79.6%      59.1%    (default)
MiniMax        MiniMax-M2.5     80.2%      -        see below
GLM (Zhipu)    glm-5            77.8%      -        see below
Kimi (Moonshot) kimi-k2.5       76.8%      -        see below
DeepSeek       deepseek-chat    73.0%      -        see below
               (V3.2)

SWE-bench = SWE-bench Verified (Feb 2026)
TB2        = Terminal-Bench 2.0 (Feb 2026)
=====

---- International ----

MiniMax        https://www.minimax.io
ANTHROPIC_BASE_URL=https://api.minimax.io/anthropic
MODEL_ID=MiniMax-M2.5

GLM (Zhipu)     https://z.ai
ANTHROPIC_BASE_URL=https://api.z.ai/api/anthropic
MODEL_ID=glm-5
```

图 5: VS Code 中打开的 `.env` 配置文件，展示了 `ANTHROPIC_API_KEY`、`MODEL_ID` 和 `BASE_URL` 的配置方式⁵

Figure 5 展示了本地配置文件中的关键内容。无论你使用的是 GLM、KIMI 还是 DeepSeek，配置方式都是一致的：只要模型服务端点兼容 Anthropic 的接口格式，就可以通过修改这三行环境变量来切换不同的模型提供商。

下面给出一个标准的环境变量配置示例：

```
1 ANTHROPIC_API_KEY=sk-...
2 ANTHROPIC_BASE_URL=https://...
3 MODEL_NAME=glm-4-plus
```

Listing 1: 环境变量示例

请

勿将真实的 API Key 硬编码并提交到公共仓库。建议在本地使用 `.env` 文件或在私有环境中设置环境变量，避免密钥泄露导致不必要的安全风险。

关于模型兼容性

GLM（智谱）、KIMI（月之暗面）、DeepSeek 等国产大模型，只要它们提供的接入端点兼容 Anthropic 的 API 格式，就可以作为本项目的底层模型使用。本质上，代码层只关心服务端点是否实现了 Anthropic SDK 所要求的请求与响应协议，而对模型内部架构没有额外限制。因此，未来出现新的兼容提供商时，也只需替换 `ANTHROPIC_BASE_URL` 和 `MODEL_NAME` 即可无缝切换。

⁵ 视频画面时间区间：00:02:56-00:03:13。

3.3 从阿里云百炼获取凭证

如果你还没有可用的 Anthropic 兼容端点,可以按照视频中的演示,从阿里云百炼平台获取 API Key 和 Base URL。具体操作如下:进入平台的 Coding Plan 页面,找到套餐专属的 API Key,以及兼容 Anthropic 接口的链接,分别复制并粘贴到本地环境变量中即可。

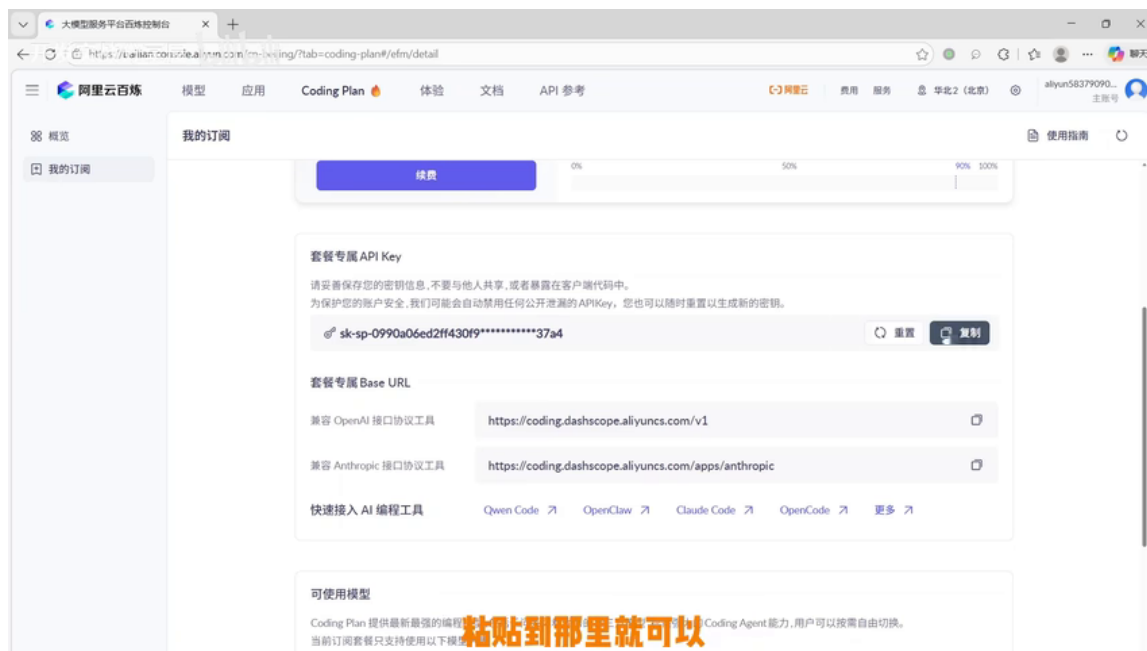


图 6: 阿里云百炼平台的 Coding Plan 页面,展示了套餐专属 API Key、兼容 Anthropic 接口的 Base URL,以及可供复制粘贴的模型接入信息⁶

如Figure 6 所示,视频中的讲师直接从阿里云百炼平台复制了 API Key 和兼容 Anthropic 接口的 Base URL。这一操作完成之后,本地环境的全部配置工作也就结束了。下一节,我们将运行第一个 Agent 示例程序,验证配置是否生效。

3.4 本章小结

本节介绍了如何根据 `learn-claude-code` 项目的 README 完成环境配置。核心要点如下:

- 按照 README Quick Start 的指引,完成仓库克隆与依赖安装;
- 必须配置三项环境变量: `ANTHROPIC_API_KEY`、`ANTHROPIC_BASE_URL` 和 `MODEL_NAME`;
- API 凭证可以从任何兼容 Anthropic 接口的平台获取,视频中以阿里云百炼平台为例进行了演示;
- GLM、KIMI、DeepSeek 等模型均受支持,只需确保服务端点兼容即可。

完成上述配置后,你就可以运行本地的 NANO Agent 了。下一章将带大家进行首次运行实战,亲眼见证一个极简 Agent 如何自主生成文件并执行程序。

⁶视频画面时间区间: 00:03:25–00:03:49。

4 首次运行 NANO Agent 实战演示

环境配置完成后，下一步就是验证这个最小化的 Agent 是否真的能够工作。本章将带领读者实际运行项目中的第一个示例脚本 `s01_agent_loop.py`，观察 NANO Agent 如何接收自然语言指令、自主生成文件并执行代码。

4.1 运行第一个脚本

根据 `learn-claude-code` 仓库 README 的 Quick Start 指引，环境就绪后的下一步就是启动 Agent。在终端中进入项目目录，执行如下命令即可运行第一个极简 Agent 示例：

```
1 python agents/s01_agent_loop.py
```

执行后，终端会进入交互模式，等待用户输入提示词。图 7 展示了该命令在终端中的启动画面。

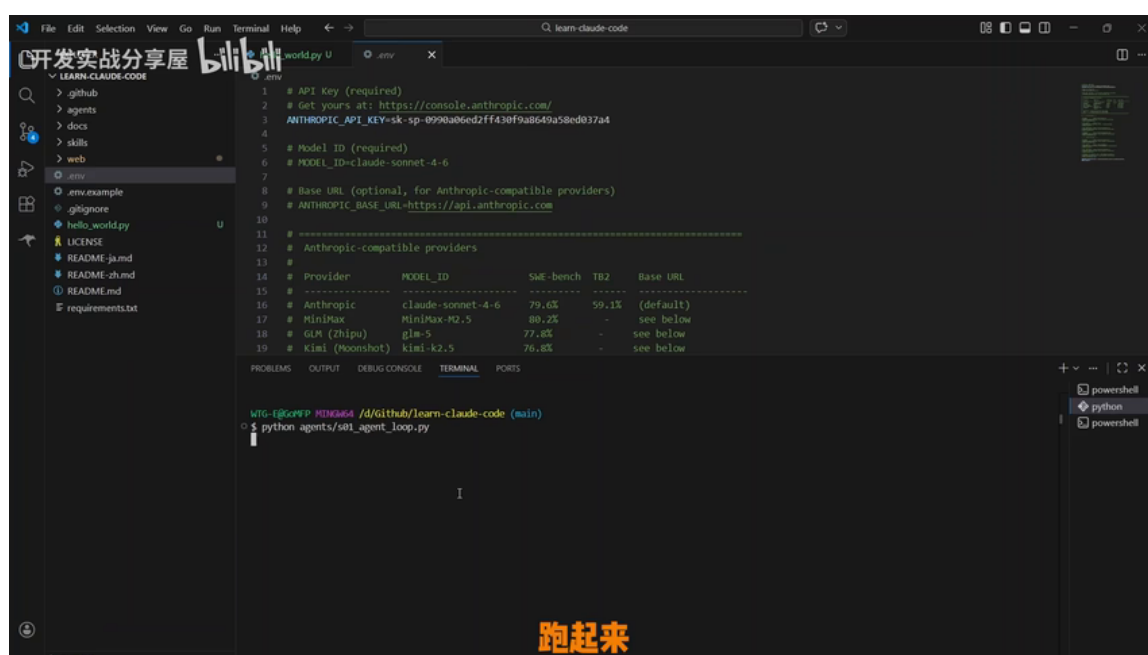


图 7: 终端中运行 `python agents/s01_agent_loop.py`，启动 NANO Agent⁷

这

个最小的脚本已经具备一个基本 Agent 的雏形。

4.2 输入相同 prompt 进行对比

为了与第 1 章中 Claude Code 的演示形成对比，我们向 NANO Agent 输入一个类似的提示词。在终端交互模式下，输入的 prompt 为：

```
1 实现一个 hello 的程序，内容是 hello hi world
```

⁷ 视频画面时间区间：00:04:28–00:04:33。

注

意一个细节：主讲人口中说的是“实现一个 hello world 的 Python 程序并跑起来”，但实际在 NANO Agent 中输入的 prompt 略有不同——他写的是“实现一个 hello 的程序，内容是 hello hi world”。此外，Claude Code 生成的文件名是 `hello_world.py`，而 NANO Agent 最终生成的文件名则是 `hello.py`。

输入提示词后，Agent 开始调用底层模型并自主决策。很快，终端中输出了文件生成与代码执行的相关日志。图 8 展示了 Agent 在终端中完成文件创建和执行的完整回显。

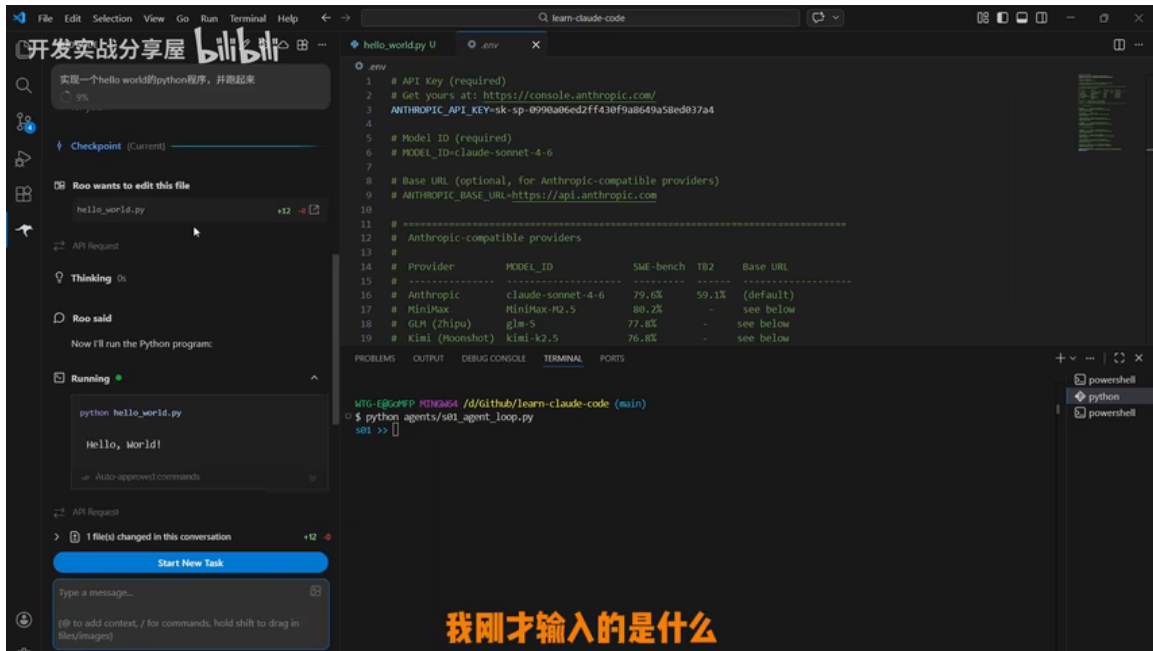


图 8: Agent 输出显示生成文件并执行成功的终端回显⁸

图 9 则更清晰地展示了最终生成的 `hello.py` 文件内容。

⁸视频画面时间区间：00:05:15–00:05:25。

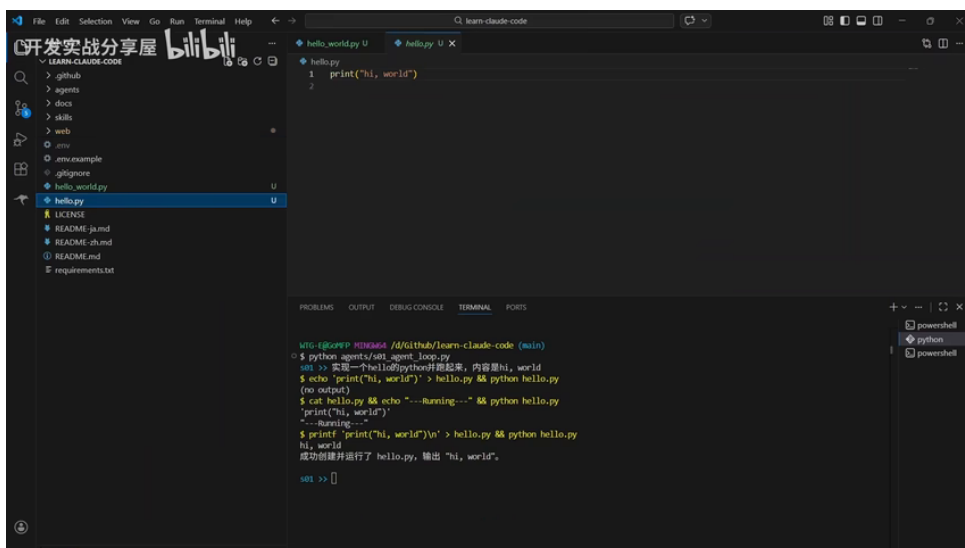


图 9: 生成的 hello.py 文件内容⁹

生成的代码非常简单，核心内容如下：

```
1 print('hi, world')
```

4.3 验证生成结果并在外部运行

虽然 Agent 在内部已经完成了运行，但为了进一步验证生成结果的正确性，我们可以在项目目录外（或直接新开一个终端）手动运行这个文件：

```
1 python hello.py
```

运行后，终端成功输出了 hi, world。图 10 记录了这一验证过程。

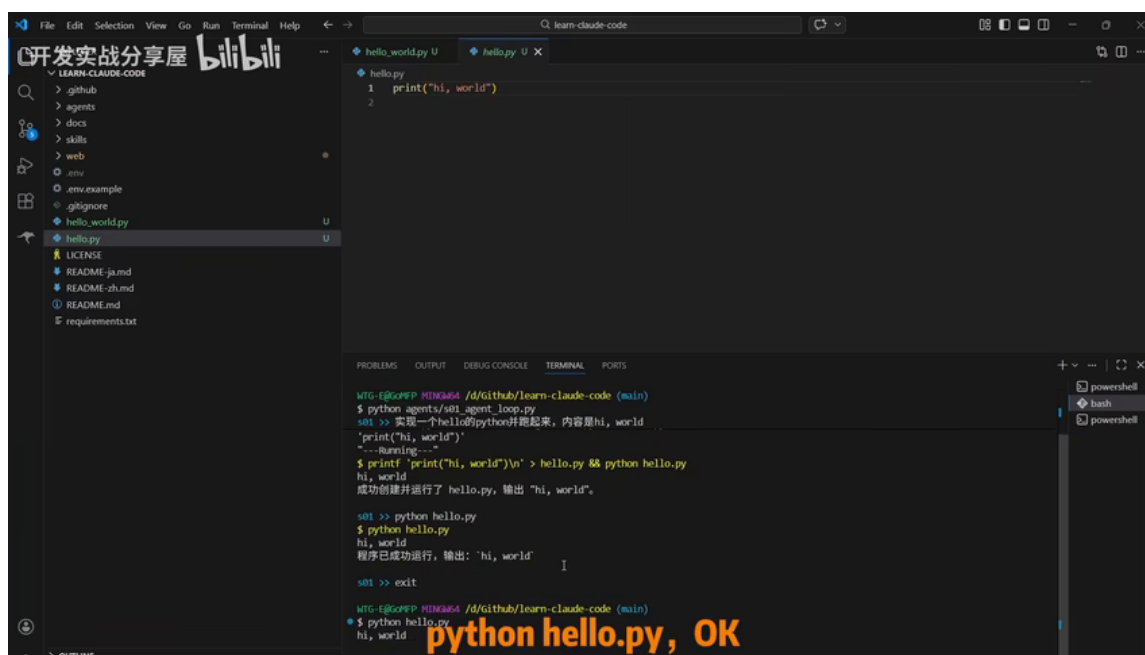


图 10: 在目录外直接运行 python hello.py，验证生成结果¹⁰

⁹视频画面时间区间：00:05:18–00:05:23。

通过这一流程可以看到，`s01_agent_loop.py` 虽然代码量不大，却已经完整覆盖了 Agent 的核心闭环：接收用户意图、生成文件、执行代码、返回结果。这意味着我们拥有了一个可以进一步扩展和学习的坚实基础。

4.4 本章小结

本章完成了 NANO Agent 的首次实战运行。从启动 `s01_agent_loop.py`，到输入自然语言提示词，再到观察 Agent 自主生成 `hello.py` 并执行成功，最后通过外部手动运行验证了结果的正确性。

关键点回顾：

- 运行 `python agents/s01_agent_loop.py` 即可启动本地极简 Agent。
- 输入自然语言 prompt 后，Agent 能够自主生成代码文件并执行。
- `s01_agent_loop.py` 已经具备一个基本 Agent 的雏形，是后续逐行阅读源码的最佳起点。

在后续章节中，我们将深入这个脚本的每一行 Python 源码，剖析 Agent 内部的循环机制、工具调用逻辑和模型交互原理。

5 总结与延伸

经过前面四章的学习，我们已经亲眼见证了一个极简 Agent 从环境配置到成功运行的完整过程。本章将对所学内容进行系统梳理，并指出后续深入源码的进阶方向。

5.1 本章小结

在这一章中，我们完成了从“手动写代码”到“与 Agent 协作”的初步体验。整个过程虽然只围绕一个最简单的 Hello World 任务展开，却已经清晰地展现了一个 Agent 与纯聊天机器人的本质区别：

- **需求输入：**我们向 Claude Code 和 NANO Agent 分别输入了自然语言提示——实现一个 `hello` 的程序，内容是 `hello hi world`。
- **自主决策：**Agent 没有止步于文本回复，而是主动切换到代码模式，生成了 `hello.py` 文件。
- **工具调用与执行：**Agent 进一步调用了文件写入与终端执行工具，最终跑通了程序，输出了预期结果。
- **源码验证：**我们还学会了如何在本地配置 API Key、Base URL 和模型名称，并成功运行了 `s01_agent_loop.py`。

这说明，`s01_agent_loop.py` 并不仅仅是一个脚本，而是一个 **Agent 雏形的起点**。它麻雀虽小，却已经具备了接收提示、做出决策、调用工具、生成文件并执行的核心闭环。

了

解 Agent 的最简结构，是后续阅读 Claude Code、Roo Code 等生产级源码的必经之路。

¹⁰ 视频画面时间区间：00:05:38–00:05:43。

5.2 后续学习路径

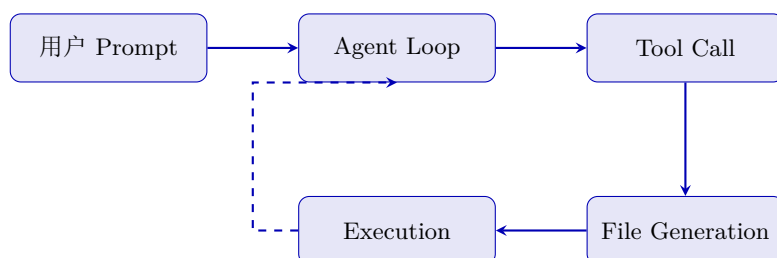
那么，从”会跑 Demo”到”真正精通 Agent”，下一步应该做什么？

答案是：逐行阅读源码。

进一步的学习方向是把 `s01_agent_loop.py` 逐行拆开，深入探讨以下几个核心问题：

1. **Agent Loop** 是如何运转的？为什么模型输出一次结果后，程序还能继续等待、继续决策、继续调用工具？
2. **Function Calling** 到底是什么？模型是如何把自然语言转化为结构化工具调用的？
3. 上下文是如何管理的？Agent 如何记住之前生成的文件内容和执行结果，并据此调整下一步动作？
4. 工具能力是如何注册的？如果我想让 Agent 拥有新建文件之外的能力（如读取文件、搜索网络），该怎么扩展？

下面这幅图可以帮助你先从宏观上把握一个极简 Agent 的工作流程：



在这个循环中，**Agent Loop** 是心脏。它源源不断地把模型的”思考”翻译为”行动”，再把行动的”结果”反馈给模型，形成自我驱动的迭代。理解了这个最简结构，你就拥有了一把打开所有生产级 Agent 框架的钥匙。

给读者的具体建议：

- 打开 learn-claude-code 仓库，定位到 `agents/s01_agent_loop.py`。
- 先通读一遍整体结构，圈出你不理解的函数和 API 调用。
- 逐行做笔记，重点关注循环体、工具列表与模型返回消息的处理逻辑。
- 尝试修改 prompt 或新增一个简单工具，观察 Agent 的行为变化。

源代码是理解 Agent 的最好教材。当你能清晰地描述”模型在什么条件下决定调用工具”、”工具结果如何影响下一次对话”时，你才真正地理解了 Agent 的运作本质。后续的学习中，我们将一起完成这段旅程。